

I. AMD64

I-a) Overview

1. AMD64 is an extension of x86, AMD64 processors **will** support legacy 32-bit x86 compatibility modes and operating systems conforming to the AMD64 ABI **may**
2. ILP32 (32-bit model) stands for int, long, and pointer are 32 bits whereas LP64 (64-bit model) stands for long and pointer are 64 bits
3. Alignment
{rule}
 1. **Structure and Union** (object size is multiple of) most strictly component alignment and each member is assigned to lowest available offset with appropriate alignment
 2. **Array** element alignment, except that a local or global array variable of length at least 16Byte or a C99 variable-length array variable has alignment of at least 16B
 3. **Bit-Fields** same size and alignment rules as other structure and union members, share **storage unit** with other members but must be contained in appropriate unit

I-b) Type

1. **size_t** unsigned [long – LP64, int – ILP32]
2. **__int128** little-endian order – 64 low-order bits at a lower address
3. **max_align_t** alignment requirement, at least as strict (as large) as that of every scalar type
4. **Boolean** when stored in integer registers (except for passing as arguments), all 8 bytes of the register are significant; any nonzero value is considered true
5. **_BigInt(N)** struct of 64-bit integer chunks if $N > 64$, otherwise same size and alignment as the smallest of char, short, int, long and long long types that can contain them

⇒ **_Alignof** queries the alignment requirement of its operand type

I-c) Register

1. 16 general purpose 64-bit, 16 SSE 128-bit and 8 x87 floating point 80-bit (64-bit in MMX/3DNow! mode) registers are global to all procedures active for a given thread
2. Intel AVX (Advanced Vector Extensions) provides 16 AVX 256-bit registers and Intel AVX-512 provides 32 SIMD 512-bit registers (%zmm*N* → AVX %ymm*N* → SSE %xmm*N*); vector register refers to either SSE, AVX or AVX-512 register
3. Intel AVX-512 also provides 8 vector mask 64-bit (%k*N*) and Intel APX (Advanced Performance Extensions) provides 16 general purpose 64-bit registers (%r16-%r31), Intel AMX (Advanced Matrix Extensions) provides 8 tile registers (%tmm*N*)

I-d) Instruction

1. AMD64 usually doesn't allow an instruction to encode 64-bit constant as immediate operand, but mostly accepts 32-bit immediate that are sign extended to the 64-bit
2. 32-bit operations with register destinations implicitly perform zero extension making loads of 64-bit immediates with upper half set to 0 even cheaper
3. Branch instructions accept 32-bit immediate operands that are sign extended and an IP relative addressing mode exists for data accesses with equivalent limitations

1) Instruction Sample

1. **CPUID**^[1] processor identification
{rule} CPU architecture information, such as vendor string, model number, internal caches' sizes, and **most** supported CPU features
2. **EMMS** Empty MMX Technology State
3. All **assembler directives** have names that begin with a period ('.')
4. The **movabs** instruction uses 64-bit addresses (AT&T syntax! just forget about that one)

2) Code Mode

1. Code models define constraints for symbolic values that allow the compiler to generate better code that improve performance and reduce code size
2. Basically code models differ in addressing (absolute versus position independent), code size, data size and address range

Small code model

1. The virtual address of code executed is known at link time
2. Symbol's addresses in $S = [0, 2^{31} - 2^{24} - 1]$ in which 24 is chosen arbitrarily that allows for all memory of objects of size up to 2^{24} to be addressed directly
3. Allow the compiler to encode symbolic references with offsets in $OS = [-2^{31}, 2^{24}]$ directly in the sign extended immediate operands
4. Allow offsets in $OZ = [0, 2^{31} - 2^{24}]$ for the zero extended immediate operands
5. Allow offsets in $OIP = [-2^{24}, 2^{24}]$ for IP relative addressing for the symbol
i.e. $symbol + offset(\%rip)$
6. **NOTE**
 1. Eventually, 32-bit immediate operand is calculated from symbol's address and offset
 2. So, symbol's address + offset must fit in 32-bit immediate!
 3. $S + OS = [-2^{31}, 2^{31} - 1]$; $S + OZ = [0, 2^{32} - 2^{25} - 1]$; $D + OIP = [-2^{31} + 1, 2^{31} - 1]$
where, $S \Rightarrow D_{IP-SYMBOL} = [-2^{31} + 2^{24} + 1, 2^{31} - 2^{24} - 1]$
 4. $2^{31} - 2^{24}$? greater than both $2^{31} - 2^{24} - 1$ and 2^{24}

Small position independent code model (PIC)

1. The virtual addresses of instructions and data are not known until dynamic link time
2. All addresses have to be relative to the instruction pointer

I-e) Addressing Mode Sample

1. **leaq 1f(%rip), %r11** where 1f means the forward label **1:** (and 1b means backward one)
-

[1]: [CUID](#)